

Lab 4 - Logic Equivalence Check (LEC)

R Sarang
EC23I2015

September 15, 2026

Objective

To understand the basics of Conformal LEC setup, and debug basic non-equivalences for the Sequential Multiplier design.

Task 1: LEC for Sequential Multiplier System

Objective

Perform a standard Logic Equivalence Check between the original RTL and the synthesized gate-level netlist to confirm that the synthesis tool correctly mapped the logic.

Implementation

The setup script (`multiplier_lec.do`) was created to read the library, read the Golden RTL files (`controller.v`, `datapath.v`, `top.v`), and read the Revised gate-level netlist (`top_gates.v`) generated by Genus.

```
1 set log file multiplier_lec.log -replace
2 read library ../lib/LIB/slow_vdd1v0_basiccells.lib -liberty -both
3 read design ../rtl/controller.v ../rtl/datapath.v ../rtl/top.v -verilog
  -golden
4 read design ../work/top_gates.v -verilog -revised
5 set system mode lec
6 add compared point -all
7 compare
8 report verification -verbose
```

Listing 1: `multiplier_lec.do`

Verification Result

The Conformal LEC tool was run using the above script. The comparison successfully mapped all 27 DFFs and primary outputs. The verification report confirms 0 Non-Equivalent points.

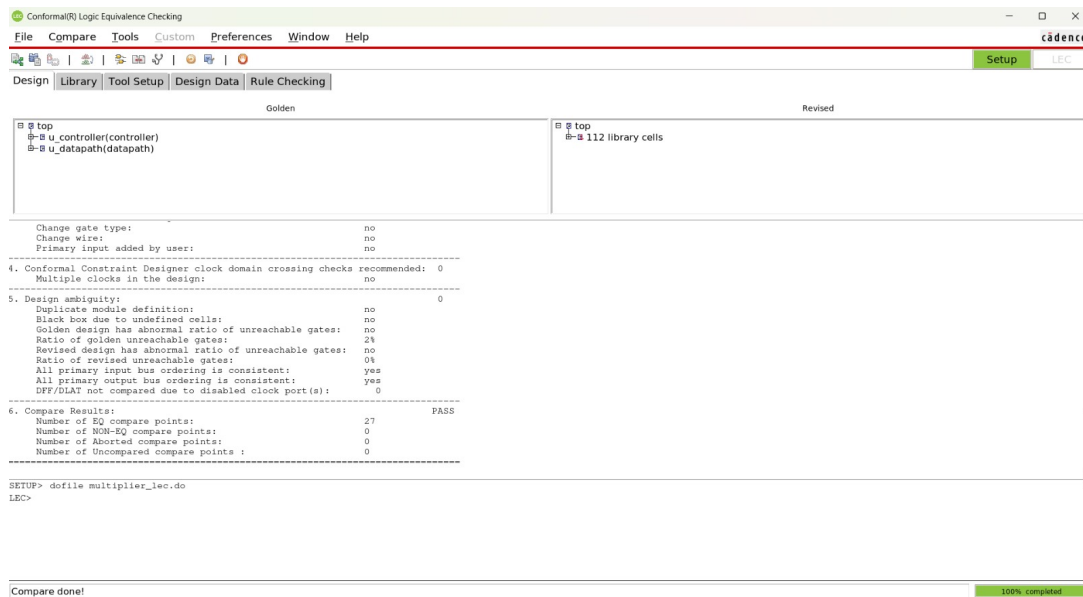


Figure 1: Task 1: LEC PASS with 0 Non-Equivalent points

Task 2: LEC for Sequential Multiplier with Clock Gating

Objective

Enable automatic clock gating insertion during synthesis for power savings, generate a report of deleted sequential elements, and successfully run LEC on the clock-gated netlist by providing the appropriate flattening constraint.

Synthesis Modification

The Genus synthesis script was modified to include clock gating. The following line was added before reading the Verilog files:

```
1 set_db / .lp_insert_clock_gating true
```

Additionally, a report was generated to track the flip-flops that were optimized out by the clock gating logic:

```
1 report_sequential -deleted > work_cg/deleted_flops.rpt
```

`work_cg/top-deleted_flops.rpt` came back empty (header only) – Genus did not remove any flip-flops outright; instead it inserted integrated clock-gating cells (`RC_CG_MOD`) ahead of the existing flops to gate their clock, rather than deleting any register.

Synthesis Comparison: Baseline vs. Clock-Gated

The clock-gated synthesis (`work_cg/top_qor.rpt`) is compared below against the Lab 2 baseline synthesis at the same 5.0ns clock period (`work/top_qor.rpt`).

Metric	Baseline (Lab 2)	Clock-Gated
Clock period	5.0 ns	5.0 ns
Critical path slack (WNS)	3105.9 ps	3184.0 ps (clk group)
TNS / Violating paths	0 / 0	0 / 0
Total cell area	200.412	181.944 (−9.2%)
Leaf instance count	76	58
Combinational instance count	58	38
Sequential instance count	18	20 (+2 clock-gating integrated cells)
Maximum fanout	18 (<code>clk</code>)	12 (<code>rc_gclk</code>)
Average fanout	2.3	2.0
Power (dynamic/leakage/total)	Not available	Not available (power analysis not enabled in this flow)

Inference: Enabling automatic clock gating reduces the synthesized cell area by about 9.2% ($200.412 \rightarrow 181.944$) and cuts the leaf instance count from 76 to 58, mainly by collapsing combinational enable/mux logic that used to feed each register’s clock-enable path ($58 \rightarrow 38$ combinational cells) into dedicated integrated clock-gating cells placed on a gated clock network. No flip-flops were deleted (see the empty `deleted_flops.rpt` above); instead 2 extra hierarchical clock-gating instances appear, which is why the sequential instance count rises slightly ($18 \rightarrow 20$). Timing is essentially unaffected – the clock-domain slack is not worse than baseline – so this area saving comes at negligible timing cost for this design. No power numbers are available to quantify the expected power benefit, since power analysis was not enabled in the supplied flow (consistent with Lab 2, Part B).

LEC without Constraint (Demonstration of Failure)

When running standard LEC on the clock-gated netlist, the verification fails because Conformal cannot map the new clock gating cells (`RC_CG_MOD`) present in the Revised netlist to the Golden RTL.

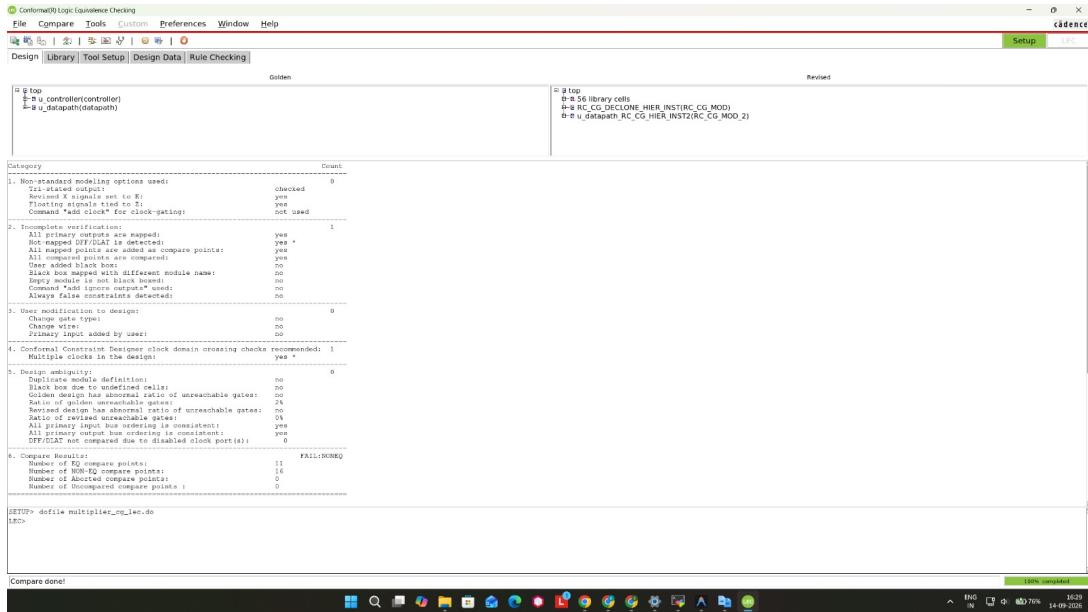


Figure 2: Task 2: LEC Failure (16 Non-Equivalent points) due to unmodeled Clock Gating

LEC with Constraint (Fix)

To instruct the tool to correctly recognize and mathematically flatten the clock gating logic, the following constraint was added to the .do file before switching to LEC mode:

```
1 set flatten model -gated_clock
```

After applying this constraint, Conformal was able to correctly compare the logic and verify equivalence.

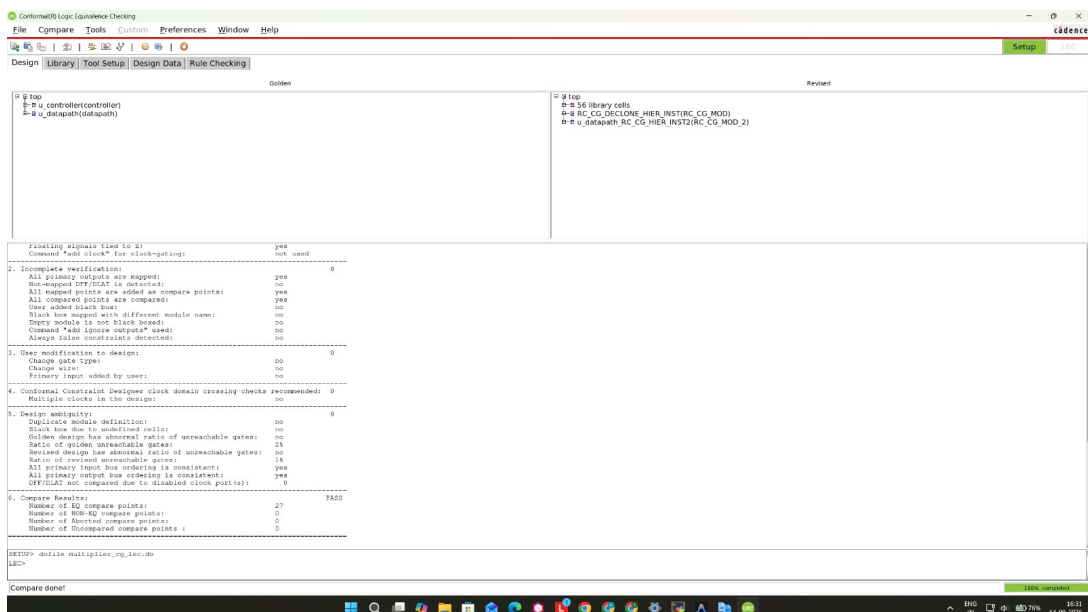


Figure 3: Task 2: LEC PASS with flattened clock gating models

Are All Flip-Flops Mapped?

Yes. The `multiplier_cg_lec.log` verification summary explicitly reports:

```
All primary outputs are mapped:  yes
Not-mapped DFF/DLAT is detected:  no
Number of NON-EQ compare points:  0
```

Every flip-flop in the Golden RTL was mapped one-to-one to a flip-flop in the Revised (clock-gated) netlist, with no unmapped DFF/DLAT points – consistent with the deleted-flops report above, which confirms Genus did not remove or merge any register during clock-gating insertion (it only added integrated clock-gating cells ahead of the existing flops). There are therefore no unmapped points to justify.

Task 3: LEC Run with a Netlist Bug

Objective

To intentionally inject a bug into the gate-level netlist and utilize Conformal LEC's graphical debug tools (Mapping Manager and Diagnosis Manager) to identify the root cause of the non-equivalence.

Bug Injection

A manual edit was made to the `top_gates_buggy.v` netlist. The input to an inverter (g1072) was incorrectly re-routed from `accumulated_prod[5]` to `accumulated_prod[4]`.

Debugging Process

Upon running LEC, the tool reported 3 Non-Equivalent compare points. Using the GUI, the following debugging steps were taken:

1. Opened the **Mapping Manager** to filter and identify the specific Non-Equivalent DFFs.
2. Right-clicked the failing point `accumulated_prod_reg[5]` and selected **Diagnose**.
3. In the **Diagnosis Manager**, Conformal's mathematical solver analyzed the logic cone and correctly identified INVX1 g1072 as the primary Error Candidate with a probability of 1.00.
4. The **Schematics** view was utilized to visually trace the boolean logic error backward from the DFF input, confirming that the Revised logic was pulling from bit [4] while the Golden reference was pulling from bit [5].

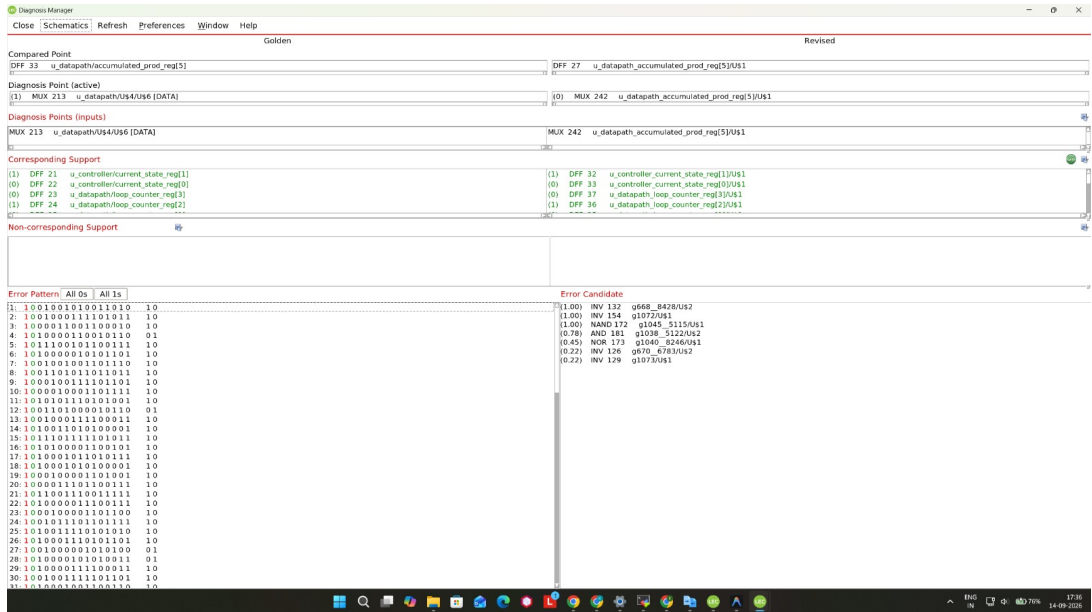


Figure 4: Task 3: Diagnosis Manager identifying g1072 with 1.00 probability

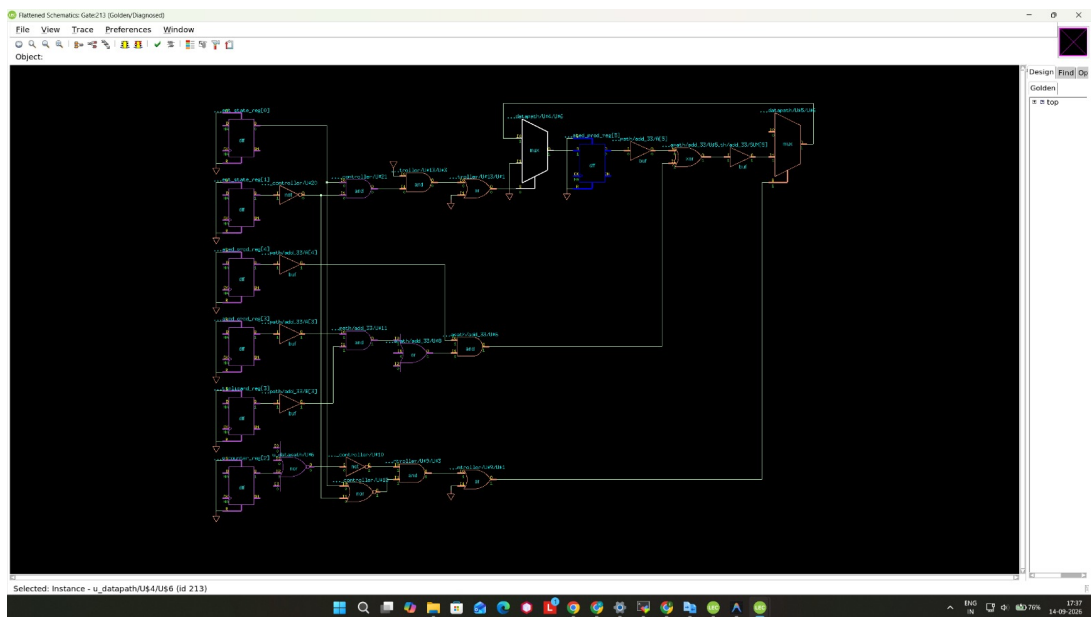


Figure 5: Task 3: Golden flattened schematic at `accumulated_prod_reg[5]`, driven correctly from bit [5]

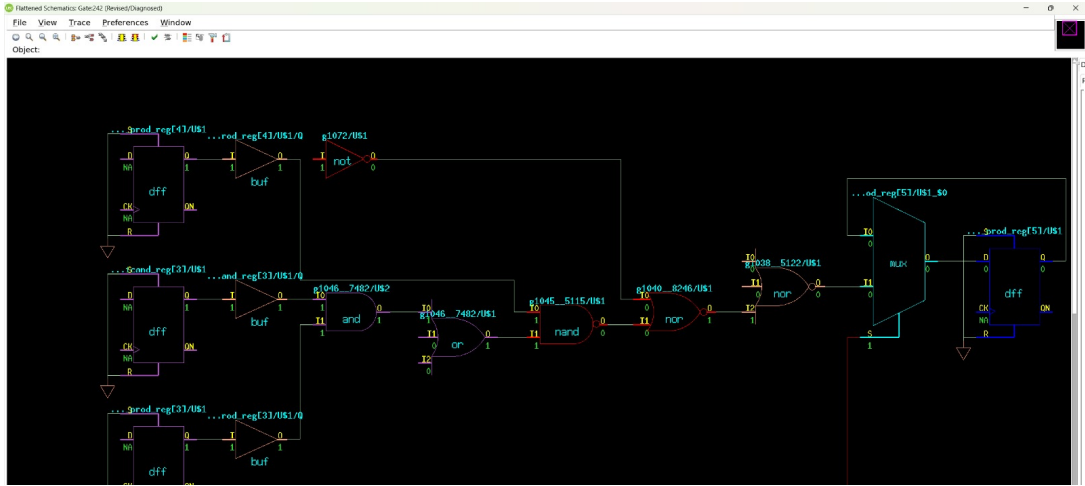


Figure 6: Task 3: Revised flattened schematic isolating the fault region around g1072

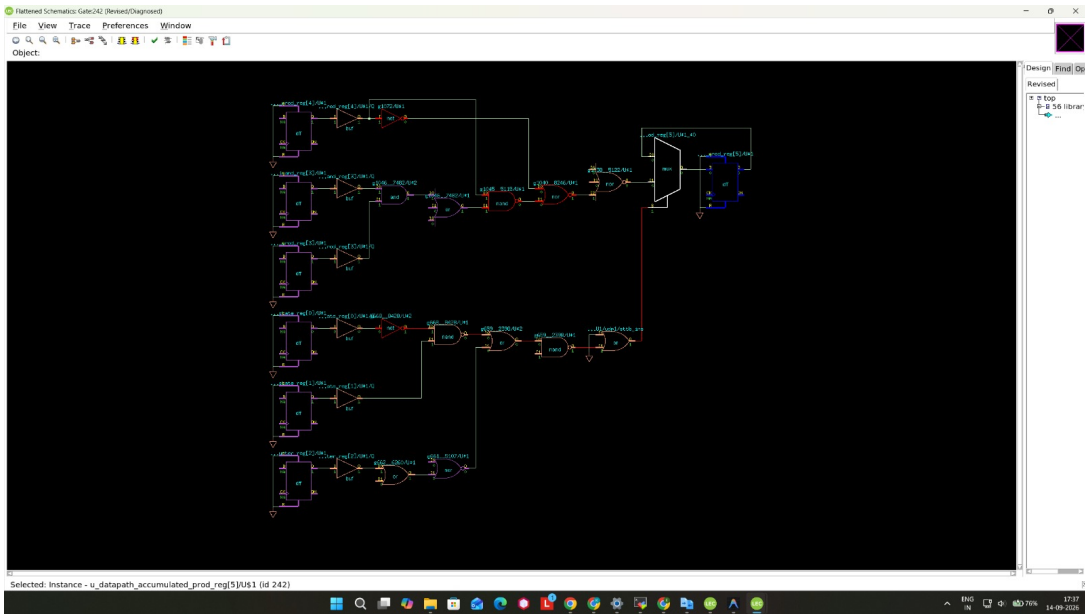


Figure 7: Task 3: Flattened Schematics tracing the error path backward to g1072, showing the incorrect re-route from bit [4]

Bug Fix and Verification

The buggy netlist was corrected by changing the wire mapping back to `accumulated_prod[5]`. A final LEC run confirmed the fix.

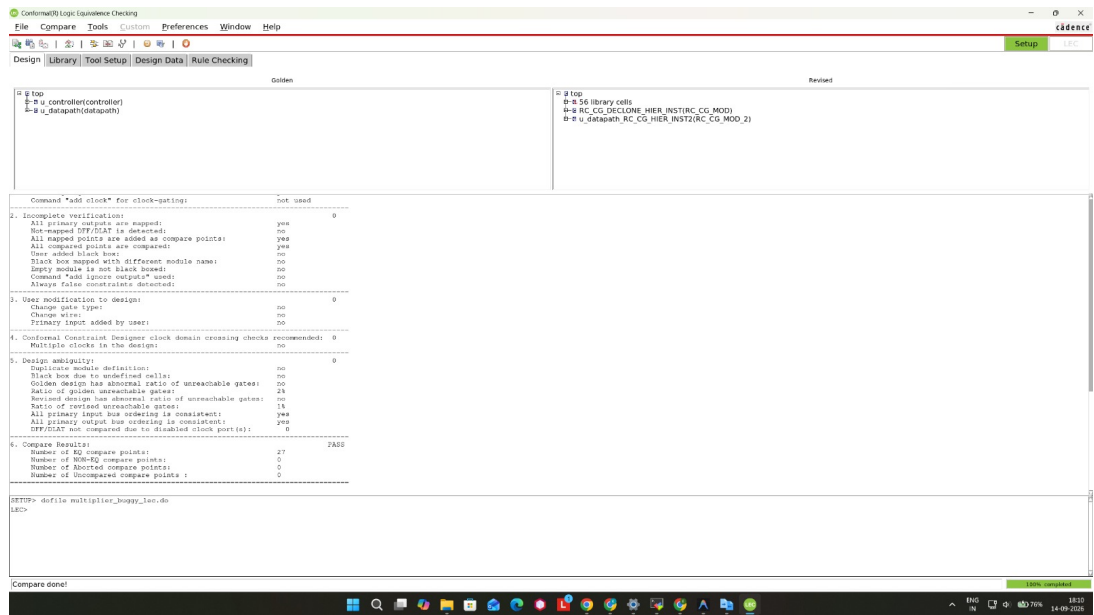


Figure 8: Task 3: Final LEC PASS after fixing the netlist bug